PROPOSTA:

Criar uma aplicação React Context que exiba as referências bibliográficas e disciplinas do curso de DSM.

Foi criado um arquivo books.json onde contém todos os dados necessários para rodar a aplicação.

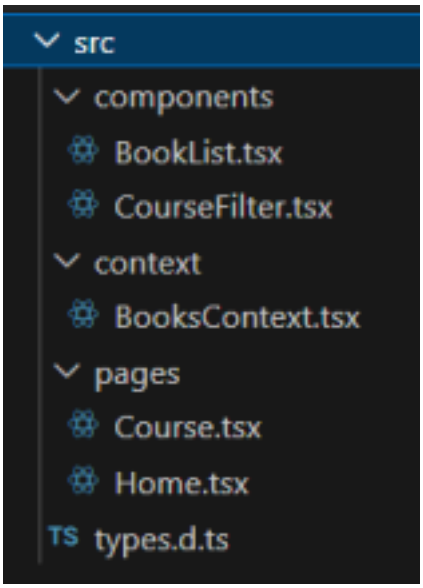
1. Estrutura do Projeto

A princípio, precisamos criar a estrutura do nosso projeto.

Escolha a pasta onde ficará o projeto e execute os seguintes comandos. Nosso caso em questão em C:

```
npx create-react-app dsm-books --template typescript
cd dsm-books
npm install @mui/material @emotion/react @emotion/styled react-router
dom axios
```

2. Pastas e arquivos sugeridos para o projeto



Podemos observar que foi criado o arquivo denominado `types.s.ts` que servirá para declarar tipos personalizados TypeScript que iremos reutilizar na aplicação. Podemos observar também que o caminho `(src/types.d.ts)` é suficiente para que o TypeScript reconheça o conteúdo automaticamente.

O conteúdo do arquivo `types.d.ts` será:

```
1  export interface Book {
2      semester: number;
3      course: string;
4      title: string;
5      author: string;
6      publisher: string;
7      year: number;
8    }
9
```

```
1   import React, { createContext, useContext, useEffect, useState } from 'react';
2   import axios from 'axios';
3   import { Book } from '../types.s';
4
5   interface BooksContextType {
6     books: Book[];
7   }
8
9   const BooksContext = createContext<BooksContextType>({ books: [] });
10
11  export const useBooks = () => useContext(BooksContext);
12
13  export const BooksProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
14    const [books, setBooks] = useState<Book[]>([]);
15
16    useEffect(() => {
17      axios.get('/data/books.json')
18        .then(response => setBooks(response.data))
19        .catch(err => console.error("Erro ao carregar books.json", err));
20    }, []);
21
22    return (
23      <BooksContext.Provider value={{ books }}>
24        {children}
25      </BooksContext.Provider>
26    );
27  };
```

2

BookList.tsx
Aula 06 – React Context

Laboratório de Desenvolvimento Web

4. Criando os componentes através do arquivo

```
import { useBooks } from "../context/BooksContext";
import { Card, CardContent, Typography } from "@mui/material";

export default function BookList() {
  const { books } = useBooks();

  return (
    <>
      {books.map((book, idx) => (
        <Card key={idx} sx={{ marginBottom: 2 }}>
          <CardContent>
            <Typography variant="h6">{book.title}</Typography>
            <Typography variant="body2">
              {book.author} - {book.publisher} ({book.year}) | Disciplina: {book.course}
            </Typography>
          </CardContent>
        </Card>
      ))}
    </>
  );
}

```


5. Podemos filtrar por disciplina através do arquivo CourseFilter.tsx

```
import { useBooks } from "../context/BooksContext";
import { MenuItem, Select, Typography } from "@mui/material";
import { useState } from "react";

export default function CourseFilter() {
  const { books } = useBooks();
  const [selected, setSelected] = useState("");

  const courses = [...new Set(books.map(book => book.course))];

  const filteredBooks = books.filter(b => selected === "" || b.course === selected);

  return (
    <>
      <Typography variant="h5">Filtrar por Disciplina</Typography>
      <Select value={selected} onChange={e => setSelected(e.target.value)} sx={{ mb: 2 }}>
        <MenuItem value="">Todas</MenuItem>
        {courses.map(course => (
          <MenuItem key={course} value={course}>{course}</MenuItem>
        ))}
      </Select>
      {filteredBooks.map((book, idx) => (
        <Typography key={idx}>{book.title} - {book.course}</Typography>
      ))}
    </>
  );
}

```

OBS → O código para buscar os livros por disciplina está incompleto. Isso faz parte da atividade que será postada no ambiente Teams.

6. Configurando as páginas com React Router

Através da Home.tsx

```
import { Typography } from "@mui/material";
import BookList from "../components/BookList";

export default function Home() {
  return (
    <>
      <Typography variant="h4" gutterBottom>Referências Bibliográficas</Typography>
      <BookList />
    </>
  );
}
```


7. Configurando Course.tsx

```
import CourseFilter from "../components/CourseFilter";

export default function Course() {
  return <CourseFilter />;
}

```

8. Integrando com rotas e contexto no App.tsx

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
import { Container, AppBar, Toolbar, Button } from '@mui/material';
import { BooksProvider } from './context/BooksContext';
import Home from './pages/Home';
import Course from './pages/Course';

export default function App() {
  return (
    <BooksProvider>
      <BrowserRouter>
        <AppBar position="static">
          <Toolbar>
            <Button color="inherit" component={Link} to="/">Início</Button>
            <Button color="inherit" component={Link} to="/cursos">Filtrar por Disciplina</Button>
          </Toolbar>
        </AppBar>
        <Container sx={{ marginTop: 4 }}>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/cursos" element={<Course />} />
          </Routes>
        </Container>
      </BrowserRouter>
    </BooksProvider>
  );
}

```

9. Servir books.json

Dentro da pasta `public` do seu projeto, vamos criar uma pasta chamada `data` e dentro dela iremos colar o arquivo `books.json` para servir a nossa aplicação.

10. Vamos ajustar o tsconfig.json

```jsx
{
  "compilerOptions": {
    "target": "ES2015",
    "lib": [
      "dom",
      "dom.iterable",
      "ES2015"
    ],
    "downlevelIteration": true,
    "allowJs": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "noFallthroughCasesInSwitch": true,
    "module": "esnext",
    "moduleResolution": "node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx"
  },
  "include": [
    "src"
  ]
}
```

Como o TypeScript utiliza o tsconfig.json para compilar o código , definimos as opções como a versão do JavaScript de saída (target), bibliotecas usadas (lib) e recursos compatíveis com navegadores mais antigos (downlevelIteration).

Utilizamos o "target": "ES2015" que irá definir o que o código final será convertido para ECMAScript 2015 (também chamado ES6). Também irá permitir o uso de recursos modernos como: let, const, Set, Map, Promise, arrow functions, etc.

Se você colocar "target": "ES5", ele tenta converter tudo para versões antigas, o que pode causar erro em recursos como Set.

Inicie o servidor utilizando o comando:

```
npm start
```

Atividade 3:

Como complemento desta aula, ajustar o código CourseFilter.tsx para realizar a busca desses livros por disciplina e por semestre. Postar no Classroom o link do projeto que você enviou no GIT.